RELAZIONE TECNICA

Implementazione dell'algoritmo di Luby con Joblib per trovare il Maximal Independent Set (MIS) di un grafo

Martina Reverberi & Sara Picardi
Corso di High Performance Computing
Docente: Prof. Michele Amoretti
a.a. 2025/2026

Indice

1 Introduzione *3*
1.1 Il problema del Maximal Independent Set 3
1.2 Algoritmo di Luby 3
1.3 Dalla Teoria alla Pratica 5
1.4 Obiettivi del progetto 5
2 Implementazione Sequenziale *6*
2.1 Struttura del progetto e moduli 6
2.2 Scelte implementative 6
2.3 Verifica di correttezza 6
2.4 Benchmark sequenziale 7
2.5 Modelli di grafo utilizzati nella sperimentazione 7
2.6 Limiti della baseline e collegamento alla parallelizzazione 8
3 Implementazione Parallela con Joblib *8*
3.1 Struttura del progetto e moduli 8
3.2 Dipendenze software 9
3.3 Obiettivo e strategia di parallelizzazione 9
3.4 Scelte Joblib 10
3.5 Descrizione del codice e flusso per round 11
3.6 Correttezza 11

3.7 Benchmark parallelo **12**
3.8 Limiti e overhead **13**
4. Analisi Prestazionale ***13***
4.1 Metodologia sperimentale **14**
4.2 Ambiente sperimentale **14**
4.3 Overhead e granularità nel MIS **14**
4.4 Scalabilità rispetto al numero di core **15**
4.5 Efficienza e Legge di Amdahl **18**
4.6 Impatto della densità del grafo **20**
5. Conclusioni e sviluppi futuri ***23***
Bibliografia ***25***

1 Introduzione

1.1 Il problema del Maximal Independent Set

Il problema del Maximal Independent Set (MIS) è uno dei problemi classici nella teoria dei grafi e riveste un ruolo centrale nello studio degli algoritmi paralleli e distribuiti.

Dato un grafo non orientato G = (V, E), un insieme **indipendente** è un sottoinsieme di nodi $I \subseteq V$ tale che per ogni coppia di nodi $(u, v) \in I$ non esista un arco $(u, v) \in E$. In altre parole, due nodi adiacenti non possono essere entrambi presenti nell'insieme.

Un insieme indipendente si dice:

- **massimale** se non è possibile aggiungere ulteriori vertici senza violare la proprietà di indipendenza;
    - Formalmente, I è un MIS se:
        - I è un insieme indipendente.
        - Ogni nodo $v \in V \setminus I$ è adiacente ad almeno un nodo in I.
- **massimo** se ha cardinalità massima tra tutti gli insiemi indipendenti del grafo.

La distinzione tra queste due nozioni è fondamentale. Il problema del *Maximum Independent Set* è noto per essere NP-hard, mentre la costruzione di un insieme indipendente massimale può essere affrontata con algoritmi efficienti, anche in ambito parallelo. ((Miller, Lecture 32))

Il MIS costituisce inoltre un importante punto di partenza per la risoluzione di numerosi altri problemi, come il matching massimale, il vertex coloring e il dominating set. Per questo motivo viene spesso considerato un vero e proprio "building block" negli algoritmi su grafi e nei sistemi distribuiti. ((Distributed Algorithms Notes, Chapter 7))

Motivazione nel contesto parallelo

Dal punto di vista sequenziale, per costruire un MIS è sufficiente scorrere i vertici in un certo ordine e inserire ciascun nodo nell'insieme se non entra in conflitto con quelli già selezionati.

La situazione cambia quando si passa a un contesto parallelo: se più processori devono decidere simultaneamente quali vertici includere, è necessario evitare che vengano scelti nodi adiacenti nello stesso momento. Coordinare queste decisioni in modo efficiente non è banale.

Nel 1986 Michael Luby propose un algoritmo parallelo randomizzato che risolve il problema in tempo logaritmico atteso ((Luby, 1986)). L'idea alla base dell'algoritmo è sorprendentemente semplice: ogni vertice prende una decisione locale basata su una priorità casuale, permettendo così una selezione simultanea senza conflitti globali.

L'algoritmo di Luby rappresenta uno dei primi esempi di utilizzo efficace della randomizzazione nella computazione parallela e costituisce ancora oggi un riferimento fondamentale nello studio degli algoritmi distribuiti.

1.2 Algoritmo di Luby

L'algoritmo proposto da Michael Luby nel 1986 rappresenta uno dei risultati più significativi nello studio degli algoritmi paralleli randomizzati. Nel lavoro originale l'analisi viene condotta nel modello EREW PRAM (Exclusive Read Exclusive Write), un modello teorico di memoria condivisa che non consente accessi concorrenti alla stessa cella. ((Luby, 1986))

In tale contesto si dimostra che il problema del Maximal Independent Set può essere risolto in tempo logaritmico utilizzando un numero polinomiale di processori, collocando il MIS tra i problemi efficientemente parallelizzabili dal punto di vista teorico.

Idea generale dell'algoritmo

L'algoritmo opera per fasi sincrone (round). A ogni iterazione, ciascun vertice ancora attivo nel grafo prende una decisione locale basata su una priorità casuale.

Durante un singolo round vengono eseguiti i seguenti passi:

1. Ogni vertice attivo assegna a sé stesso una priorità casuale (tipicamente un numero reale uniforme in [0,1]).
2. Un vertice viene selezionato se la sua priorità è strettamente minore rispetto a quella di tutti i suoi vicini attivi.
3. I vertici selezionati vengono aggiunti al MIS.
4. Tali vertici e tutti i loro vicini vengono rimossi dal grafo residuo.
5. Il processo continua finché non rimangono più vertici attivi.

Formalmente, se r(v) è la priorità casuale associata al vertice v, allora:

$$v \in MIS \text{ se } r(v) < r(w)\ \forall w \in N(v)$$

La decisione è completamente locale: ogni vertice necessita esclusivamente delle informazioni relative ai propri vicini.

Correttezza

La correttezza dell'algoritmo discende direttamente dalla regola di selezione:

- Due vertici adiacenti non possono essere selezionati nello stesso round.
- Una volta selezionato un vertice, tutti i suoi vicini vengono rimossi.
- Alla terminazione, ogni vertice non incluso è adiacente a un vertice del MIS.

L'insieme ottenuto è quindi indipendente e massimale.

Complessità teorica

L'analisi probabilistica mostra che, in valore atteso, a ogni round viene eliminata una frazione costante degli archi del grafo. Ne consegue che il numero di round è O(log n) con alta probabilità. ((Miller, Lecture 32))

Nel modello PRAM ideale, ciò si traduce in un tempo parallelo logaritmico con un numero polinomiale di processori.

1.3 Dalla Teoria alla Pratica

Sebbene l'algoritmo di Luby sia stato analizzato nel modello PRAM, tale modello assume condizioni ideali di parallelismo che non riflettono completamente l'esecuzione su architetture multicore reali.

In un contesto di High Performance Computing intervengono fattori aggiuntivi, come:

- costi di sincronizzazione tra thread o processi;
- overhead di creazione e gestione dei task;
- costi di comunicazione e trasferimento dei dati;
- limitazioni del Global Interpreter Lock (GIL) nel caso di Python.

Nel presente progetto, particolare attenzione è stata posta alla riduzione di tali overhead. In una prima versione dell'implementazione, l'utilizzo di strutture dati Python (dizionari e liste) comportava costi elevati di serializzazione e comunicazione tra i processi, limitando fortemente le prestazioni della versione parallela.

Per mitigare questi effetti, il grafo è stato successivamente convertito in una rappresentazione compatta basata su array NumPy, simile al formato CSR (Compressed Sparse Row). Questa scelta consente:

- accessi più efficienti alla struttura del grafo;
- riduzione del costo di serializzazione;
- migliore utilizzo della memoria e della cache.

La parallelizzazione è stata implementata tramite la libreria Joblib, sfruttando principalmente il backend a processi (loky). In questo contesto, la fase più naturalmente parallelizzabile dell'algoritmo è la selezione dei vertici con priorità minima locale, mentre le operazioni di aggiornamento del grafo residuo rimangono in gran parte sequenziali.

Di conseguenza, le prestazioni complessive dipendono non solo dal numero di core disponibili, ma anche dall'equilibrio tra:

- lavoro effettivamente parallelizzabile;
- overhead introdotto dal modello di esecuzione.

1.4 Obiettivi del progetto

Alla luce di queste considerazioni, il progetto si propone di:

1. sviluppare un'implementazione sequenziale dell'algoritmo di Luby come baseline di riferimento;
2. parallelizzare l'algoritmo utilizzando la libreria Joblib;
3. confrontare sperimentalmente le due versioni, analizzando:
    - speedup rispetto alla versione sequenziale;
    - scalabilità al variare del numero di core;
    - impatto dell'overhead di parallelizzazione;
    - comportamento su grafi di diversa dimensione e densità.

L’obiettivo finale è valutare in modo critico la distanza tra il modello teorico ideale e l’esecuzione reale su architetture multicore, evidenziando vantaggi e limiti dell’approccio adottato.

2 Implementazione Sequenziale

Al fine di disporre di un termine di confronto per la versione parallela, è stata sviluppata un'implementazione sequenziale dell'algoritmo di Luby in linguaggio Python.
Questa versione costituisce la baseline sperimentale rispetto alla quale verranno valutati speedup, scalabilità ed efficienza della successiva implementazione parallela con Joblib.

2.1 Struttura del progetto e moduli

Il codice è organizzato in moduli separati, in modo da tenere distinta la parte algoritmica dalla generazione dei grafi, dalla validazione e dal benchmarking:

- **luby_seq.py**: implementazione della baseline sequenziale e conteggio dei round
- **graph_utils.py**: generatori di grafi (Erdős–Rényi, path, star) e controllo di non-orientamento
- **validate.py**: funzioni di verifica di indipendenza e massimalità
- **tests_seq.py**: test automatici su grafi piccoli, strutturati e random
- **bench_seq.py**: campagna sperimentale e salvataggio risultati in CSV
- **plot_seq.py**: generazione automatica dei grafici a partire dai CSV

Questa migliora la manutenibilità del codice e consente di isolare chiaramente la componente algoritmica dalle attività di analisi sperimentale

2.2 Scelte implementative

Rappresentazione del grafo.
Il grafo è una lista di adiacenza rappresentata come dizionario *Dict[int, Set[int]]*, cioè per ogni nodo si mantiene l’insieme dei vicini. Questa struttura rende immediati sia il controllo dei vicini attivi sia le operazioni di intersezione.

Randomizzazione e riproducibilità.
L’algoritmo è randomizzato, quindi la riproducibilità è gestita tramite un *seed* esplicito: all’inizio dell’esecuzione si crea un generatore *random.Random(seed)*, così da ottenere risultati replicabili.

Tie-break deterministico.
Nel confronto tra priorità, la priorità associata a un nodo non è solo *rng.random()*, ma una coppia *(rng.random(), v)*. In questo modo, anche se (raramente) due nodi ottenessero lo stesso numero casuale, il confronto resta deterministico grazie all’id del nodo.

Conteggio dei round.
La funzione può opzionalmente restituire anche il numero di iterazioni del ciclo principale tramite *return_rounds=True*, utile per l’analisi sperimentale.

2.3 Verifica di correttezza

Per evitare errori “silenziosi”, la correttezza del risultato viene controllata automaticamente con due proprietà: indipendenza e massimalità, implementate nel modulo *validate.py*.

- **Indipendenza**: nessun nodo del MIS è adiacente a un altro nodo del MIS. Implementata verificando che, per ogni nodo selezionato, l’intersezione tra i suoi vicini e l’insieme sia vuota.
- **Massimalità**: per ogni nodo non selezionato deve esistere almeno un vicino nel MIS; altrimenti il MIS sarebbe estendibile.

Inoltre, è stata sviluppata una piccola suite di test automatici (*tests_seq.py*) che applica l’algoritmo a diverse tipologie di grafi e a più valori di seed, al fine di verificare la robustezza dell’implementazione rispetto alla randomizzazione.

La separazione tra validazione funzionale e benchmark prestazionale consente di garantire la correttezza dell’algoritmo prima di procedere con l’analisi dei tempi di esecuzione.

2.4 Benchmark sequenziale

La campagna di benchmark è gestita da *bench_seq.py.* Per ogni coppia (n, p) viene generato un grafo Erdős–Rényi, viene effettuato un breve warm-up e poi si ripete l’esecuzione più volte variando il seed.

Per ogni configurazione vengono registrati:

- tempo medio e deviazione standard,
- dimensione del MIS (media/min/max),
- numero di round (media/min/max),
- esito dei controlli di correttezza.

I risultati sono salvati in formato CSV e successivamente visualizzati tramite *plot_seq.py.*

Le dimensioni considerate coprono più ordini di grandezza:

- grafi piccoli: $n \in \{200,500,1000\}$
- grafi medi: $n \in \{5000,10000\}$
- grafi grandi: $n \in \{50000,100000,250000,500000\}$

con valori di densità p variabili.

2.5 Modelli di grafo utilizzati nella sperimentazione

Per la valutazione sperimentale dell’algoritmo sono stati considerati sia grafi strutturati sia grafi casuali, in modo da analizzare il comportamento dell’implementazione in scenari differenti.

In particolare, sono stati utilizzati:

- **Grafo “toy” (triangolo + nodo isolato)**: impiegato per verificare rapidamente la correttezza dell’implementazione e la gestione di casi limite, come la presenza di nodi isolati.

- **Path (grafo lineare)**: rappresenta un grafo sparso e regolare, con grado massimo pari a 2. Questo modello consente di osservare il comportamento dell'algoritmo in presenza di una struttura semplice e prevedibile, in cui la dimensione del MIS è circa n/2.
- **Star (grafo a stella)**: utilizzato come caso significativo in cui esistono insiemi indipendenti massimali di cardinalità molto diversa. In una stella, ad esempio, è possibile ottenere un MIS composto dal solo nodo centrale oppure da tutte le foglie. Questo caso evidenzia la differenza tra maximal e maximum independent set e permette di osservare la variabilità introdotta dalla randomizzazione.
- **Grafi Erdős–Rényi G(n,p):** Costituiscono il modello principale per l'analisi quantitativa. Per ovviare al problema del timeout computazionale nella generazione di reti Extra-Large (es. n=500.000), che richiederebbe un tempo quadratico O(n^2) con un approccio standard, si è implementato nativamente l'algoritmo di generazione ottimizzata di Batagelj e Brandes (2005). Sfruttando la distribuzione geometrica delle distanze tra gli archi, questa implementazione abbatte la complessità a O(n+m), consentendo lo studio sistematico della scalabilità anche sui grandi cluster. Variando n (numero di nodi) e p (probabilità di arco) è possibile controllare dimensione e densità del grafo, studiandone l'impatto sul numero di round e sui tempi di esecuzione. La combinazione di grafi strutturati e casuali permette quindi di coprire sia casi limite sia scenari statistici generali, fornendo una valutazione sperimentale più completa e significativa.

2.6 Limiti della baseline e collegamento alla parallelizzazione

La versione sequenziale esegue tutte le operazioni in un unico processo, rendendola un riferimento naturale per la valutazione delle prestazioni.

Dal punto di vista algoritmico, le operazioni principali di ciascun round sono:

- assegnazione delle priorità casuali;
- verifica della condizione di minimo locale per ciascun nodo;
- aggiornamento del grafo residuo.

Tra queste, la verifica della condizione di minimo locale è la fase più naturalmente parallelizzabile, poiché ogni nodo può essere trattato in modo indipendente.

Tuttavia, la rappresentazione basata su strutture Python (dizionari e insiemi), pur essendo adatta al caso sequenziale, introduce overhead significativi in un contesto parallelo, in particolare in termini di accesso non contiguo alla memoria e costi di gestione degli oggetti Python.

Questa osservazione motiva la scelta, nella versione parallela, di adottare una rappresentazione più compatta e orientata agli array, al fine di migliorare l'efficienza e ridurre i costi di parallelizzazione.

3 Implementazione Parallela con Joblib

3.1 Struttura del progetto e moduli

La versione parallela dell'algoritmo mantiene la stessa organizzazione modulare adottata per la baseline sequenziale, ma introduce moduli specifici per la gestione del parallelismo tramite Joblib.

In particolare, la parte parallela è organizzata nei seguenti file:

- **luby_par.py**: implementazione parallela dell'algoritmo di Luby tramite Joblib, con parallelizzazione della selezione dei minimi locali all'interno di ciascun round.
- **tests_par.py**: suite di test automatici per verificare la correttezza dell'implementazione parallela su grafi toy, path, star e grafi Erdős–Rényi.
- **bench_par.py**: esecuzione della campagna di benchmark parallela, con variazione del numero di worker (*n_jobs*) e raccolta dei risultati in file CSV.
- **plot_par.py**: generazione automatica dei grafici relativi alla versione parallela (tempo medio, round medi, dimensione del MIS) al variare dei parametri sperimentali.
- **plot_compare.py**: script utilizzato per confrontare direttamente la versione sequenziale e quella parallela, producendo grafici di **speedup** ed **efficienza**.

I moduli relativi alla generazione dei grafi (*graph_utils.py*) e alla validazione della correttezza (*validate.py*) sono condivisi tra le due implementazioni, garantendo coerenza nel confronto sperimentale.

Questa organizzazione modulare consente di mantenere separati:

- l'algoritmo,
- la validazione,
- il benchmarking,
- la visualizzazione dei risultati,

facilitando sia il testing sia l'analisi prestazionale.

3.2 Dipendenze software

L'implementazione del progetto utilizza alcune librerie Python per la gestione del parallelismo e per l'analisi dei risultati sperimentali.

In particolare sono state utilizzate le seguenti dipendenze:

- **Joblib**, utilizzata per implementare la versione parallela dell'algoritmo sfruttando più core della CPU;
- **Pandas**, impiegata per la gestione e l'analisi dei risultati sperimentali salvati nei file CSV;
- **Matplotlib**, utilizzata per la generazione dei grafici dell'analisi prestazionale.

L'esecuzione dei benchmark è stata effettuata sul cluster HPC dell'Università di Parma, dove la libreria **Joblib** è stata installata nell'ambiente utente locale.
La generazione dei grafici e l'analisi dei risultati sono state invece eseguite in ambiente locale sul computer personale, utilizzando i file CSV prodotti durante le esecuzioni sul cluster.

3.3 Obiettivo e strategia di parallelizzazione

L'obiettivo della versione parallela è sfruttare il parallelismo disponibile su architetture multicore per ridurre il tempo di esecuzione dell'algoritmo di Luby, mantenendo invariata la correttezza del risultato. L'obiettivo è ridurre il tempo di esecuzione sfruttando il parallelismo di dati (Data Parallelism). Per superare i limiti di scalabilità evidenziati nella baseline, la versione parallela implementa una trasformazione radicale della struttura dati: il grafo viene convertito da un dizionario di oggetti Python a una **rappresentazione contigua CSR-like (Compressed Sparse Row)** tramite la funzione interna _convert_to_csr_like.

Vengono utilizzati due array NumPy:

- indices: contiene la concatenazione di tutte le liste di adiacenza.
- indptr: contiene i puntatori (offset) che delimitano i vicini di ogni nodo.

Questa struttura è fondamentale in ambito HPC poiché garantisce la **località dei dati** e permette l'accesso agli archi con complessità O(1) in memoria contigua.

L'algoritmo è strutturato in round; all'interno di ciascun round, per ogni nodo attivo vengono eseguite operazioni concettualmente indipendenti:

- assegnazione di una priorità casuale;
- verifica della condizione di minimo locale rispetto ai vicini.

In particolare, la verifica della condizione:

$$r(v) < r(w) \ \forall w \in N(v)$$

è completamente locale e rappresenta la parte più naturalmente parallelizzabile.

La strategia adottata consiste quindi nel parallelizzare la fase di selezione dei minimi locali, mantenendo invece sequenziali:

- la generazione delle priorità;
- l'aggiornamento del grafo residuo;
- la rimozione dei nodi selezionati.

Per ridurre l'overhead del parallelismo, il grafo viene convertito in una rappresentazione compatta basata su array NumPy, simile al formato CSR (Compressed Sparse Row). Questa scelta consente di migliorare la località dei dati e ridurre i costi di comunicazione tra i worker.

3.4 Scelte Joblib

La parallelizzazione è implementata tramite la libreria Joblib, utilizzando l'interfaccia *Parallel* e la funzione *delayed.*

A differenza di una prima implementazione basata su strutture Python (dizionari e insiemi), che comportava elevati costi di serializzazione, la versione finale utilizza una rappresentazione compatta del grafo tramite array NumPy. Una scelta critica per la scalabilità su grafi "Extra-Large" è stata l'integrazione di **NumPy con la Shared Memory**. Poiché i grafi da 500.000 nodi occupano una porzione significativa di RAM, l'invio di copie del grafo a ogni worker causerebbe un crash per Out-of-Memory. Sfruttando array NumPy e il backend loky, Joblib implementa il **Memory Mapping (memmap)**: gli array del grafo risiedono in una zona di memoria condivisa a cui i worker accedono in sola lettura, eliminando l'overhead di copia e riducendo drasticamente il consumo di RAM.

Il carico di lavoro viene suddiviso in chunk di dimensione simile. Ogni worker esegue una funzione che:

- riceve un sottoinsieme di nodi;

- verifica la condizione di minimo locale;
- restituisce i nodi selezionati.

I risultati parziali vengono quindi combinati in modo sequenziale.

Per garantire la riproducibilità:

- le priorità vengono generate con un generatore pseudo-casuale inizializzato con seed;
- l'ordine dei nodi è deterministico.

3.5 Descrizione del codice e flusso per round

Il seguente flusso corrisponde alla logica implementata nella funzione *luby_joblib* del file *luby_par.py*, che gestisce l'esecuzione dell'algoritmo round dopo round.

Ogni round della versione parallela segue le seguenti fasi:

1. Costruzione della struttura indicizzata (array NumPy) dei nodi attivi.
2. I nodi attivi vengono ordinati per garantire determinismo nell'assegnazione delle priorità.
3. Generazione di array NumPy contenente le priorità.
4. Suddivisione dei nodi in chunk.
5. Esecuzione parallela del controllo di minimo locale.
6. Unione dei risultati parziali.
7. Aggiornamento del MIS globale.
8. Rimozione dei nodi selezionati e dei loro vicini dall'insieme attivo.

Dal punto di vista logico, l'algoritmo rimane identico alla versione sequenziale; cambia esclusivamente la modalità di valutazione della condizione locale.

3.6 Correttezza

La correttezza della versione parallela viene verificata utilizzando le stesse funzioni della baseline sequenziale (*is_independent_set e is_maximal*), definite nel modulo *validate.py.*

Questo approccio consente di garantire che la parallelizzazione non introduca errori logici rispetto alla versione sequenziale di riferimento.

È stata inoltre implementata una suite di test (*tests_par.py*) che applica l'algoritmo parallelo a diverse tipologie di grafi:

- grafi toy;
- grafi path;
- grafi star;
- grafi Erdős–Rényi.

I test vengono eseguiti per diversi valori di seed e verificano le seguenti proprietà:

- **indipendenza**: nessuna coppia di nodi nel MIS è adiacente;
- **massimalità**: ogni nodo non selezionato è adiacente a un nodo del MIS;
- **terminazione corretta**, verificata tramite un numero di round positivo.

Poiché la parallelizzazione riguarda esclusivamente la fase di selezione dei minimi locali, mentre l'aggiornamento del grafo residuo rimane sequenziale, la correttezza globale dell'algoritmo è preservata.

3.7 Benchmark parallelo

La campagna sperimentale è implementata nel file *bench_par.py*, che esegue i benchmark variando il numero di worker (*n_jobs*) e raccoglie i risultati in file CSV.

Per ogni configurazione sperimentale:

- viene eseguito un warm-up non misurato;
- l'algoritmo viene eseguito più volte con seed differenti, al fine di ridurre la variabilità dovuta alla randomizzazione.

Per ciascuna configurazione:

Parametro	Small	Medium	Large
n	200, 500, 1000	5000, 10000	50000, 100000, 250000, 500000
p	0.01, 0.05, 0.1	0.01, 0.05	0.0005, 0.01
n_jobs	1, 2, 4, 8	1, 2, 4, 8	1, 2, 4, 8
Ripetizioni	4	3	2
Backend	loky	loky	loky

Tabella 1

vengono registrate le seguenti metriche:

- **tempo medio di esecuzione** e deviazione standard.
- **dimensione del MIS** (media, minimo e massimo).
- **numero medio di round**.
- **speedup**, definito come S = $\frac{Tseq}{Tpar}$ dove Tseq è ottenuto dalla baseline sequenziale (*seq_results.csv*).
- **efficienza**, definita come:

$$E = \frac{S}{n_jobs}$$

Oltre a queste metriche globali, l'implementazione raccoglie anche statistiche interne (come il tempo speso nelle diverse fasi dell'algoritmo), utilizzate per analizzare il contributo delle diverse componenti al tempo totale.

I risultati vengono salvati in *par_results.csv* e successivamente elaborati per la generazione dei grafici di analisi prestazionale (tempo, speedup, efficienza e breakdown dei tempi).

3.8 Limiti e overhead

Nonostante l'algoritmo di Luby sia altamente parallelizzabile nel modello PRAM, l'implementazione reale presenta diversi fattori limitanti.

I principali sono:

- **Overhead di parallelizzazione**: creazione dei processi, scheduling e sincronizzazione introdotti da Joblib;
- **Costo di comunicazione dei dati**: sebbene ridotto tramite l'uso di NumPy, rimane non nullo;
- **Parte sequenziale residua**, in particolare:
  - aggiornamento del grafo residuo;
  - fusione dei risultati;
- **Granularità del lavoro**: per grafi piccoli, il lavoro per ciascun worker è insufficiente a compensare l'overhead;
- **Accesso alla memoria**: il problema è in parte memory-bound, limitando la scalabilità.

Nelle implementazioni iniziali, il Global Interpreter Lock (GIL) limitava il parallelismo nelle operazioni Python. Al contrario, nella versione finale del progetto l'utilizzo di NumPy riduce questo effetto grazie all'esecuzione in codice nativo.

Di conseguenza, lo speedup osservato non è lineare rispetto al numero di core ed è coerente con la legge di Amdahl:

$$S(n) = \frac{1}{(1-p) + p/n}$$

dove p rappresenta la frazione parallelizzabile dell'algoritmo.

In particolare:

- per grafi piccoli, l'overhead domina;
- per grafi grandi, il parallelismo diventa più efficace, ma resta limitato dalla componente sequenziale e dai costi di gestione.

4. Analisi Prestazionale

L'obiettivo di questa sezione è valutare il comportamento dell'algoritmo di Luby dal punto di vista delle prestazioni, confrontando la versione sequenziale con quella parallela e analizzando l'impatto di diversi fattori, quali la dimensione del grafo, la densità degli archi e il numero di core utilizzati.

L'analisi prestazionale deve tenere conto che l'algoritmo di Luby è intrinsecamente **Memory-Bound**. Poiché ogni worker deve accedere casualmente agli indici della active_mask e delle priorità per verificare i vicini, la velocità è limitata dalla banda passante della memoria e dai *cache miss*. L'adozione del formato CSR è stata la contromisura principale per mitigare questo limite,

compattando i dati e migliorando il riutilizzo della cache L3 del processore durante la scansione dei chunk.

4.1 Metodologia sperimentale

Per analizzare le prestazioni dell'algoritmo sono stati eseguiti benchmark su grafi generati secondo il modello Erdős–Rényi G(n, p), in cui ogni possibile arco è presente con probabilità p.

Gli esperimenti sono stati condotti variando i seguenti parametri:

- il numero di nodi n;
- la probabilità di connessione p;
- il numero di worker n_jobs.

Per ogni configurazione sperimentale (Tabella 1) sono state eseguite più esecuzioni indipendenti utilizzando seed differenti, al fine di ridurre la variabilità introdotta dalla natura randomizzata dell'algoritmo.

I tempi riportati nei grafici rappresentano il valore medio calcolato sulle diverse esecuzioni.

4.2 Ambiente sperimentale

Gli esperimenti sono stati eseguiti sul cluster HPC dell'Università di Parma, utilizzando il sistema di scheduling SLURM per la gestione delle risorse computazionali.

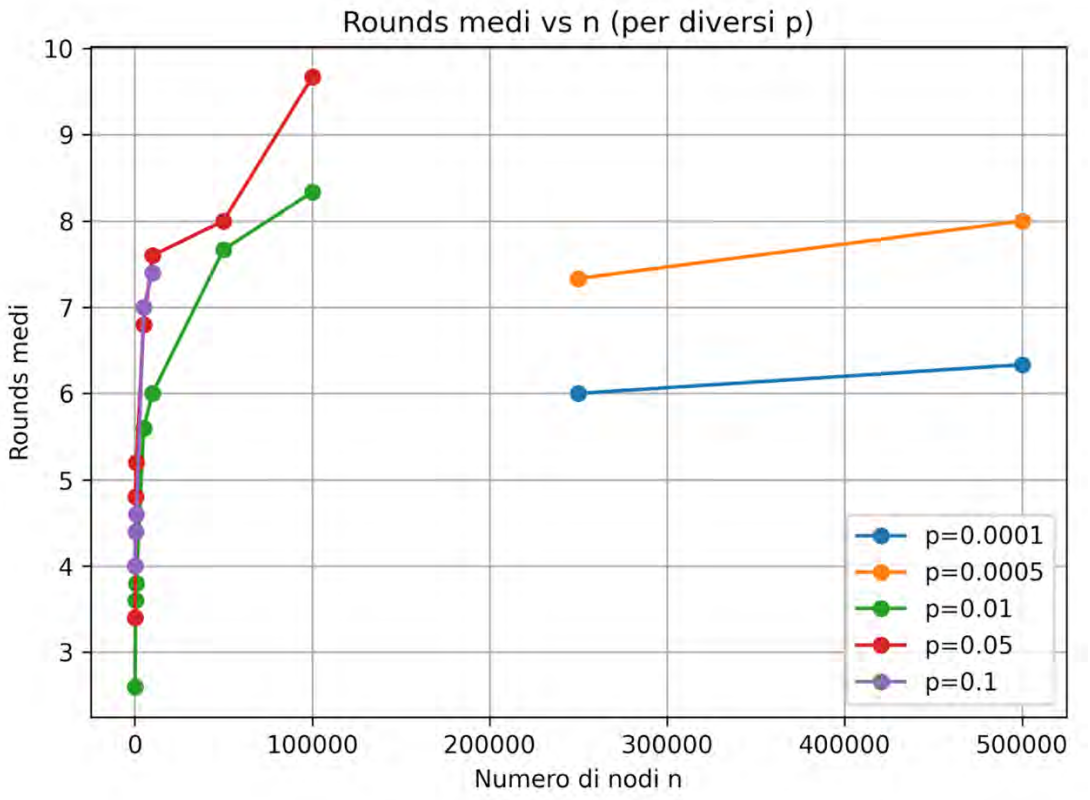
Ogni job è stato configurato per richiedere un'allocazione dedicata di 16 CPU e 64GB di RAM, garantendo che l'overhead di sistema non influenzasse le misure di tempo.

L'implementazione dell'algoritmo è stata realizzata in Python; la versione parallela utilizza la libreria Joblib con backend a processi (loky); tuttavia, le prestazioni osservate risultano fortemente influenzate dall'overhead di gestione e dai costi di comunicazione, che limitano il parallelismo effettivo.

4.3 Overhead e granularità nel MIS

L'algoritmo di Luby procede per **round iterativi**. In ogni round i nodi attivi assegnano una priorità casuale e verificano se rappresentano un minimo locale rispetto ai propri vicini. I nodi selezionati vengono aggiunti al MIS e rimossi insieme ai loro vicini.

Il numero di round è quindi un parametro fondamentale per comprendere il comportamento complessivo dell'algoritmo. In un'architettura parallela, ogni round funge da **barriera di sincronizzazione globale**: i worker devono terminare la fase di selezione prima che il processo principale possa aggiornare lo stato del grafo e lanciare l'iterazione successiva.



Numero medio di round in funzione del numero di nodi del grafo per diversi valori di p.

Osservando il grafico, si nota innanzitutto che tutte le curve crescono molto lentamente all'aumentare del numero di nodi n.

Per valori piccoli di n, il numero medio di round è contenuto (circa 3–6), mentre aumentando la dimensione del grafo le curve crescono sempre più lentamente: anche per n=100000 o n=500000, il numero di round rimane inferiore a 10.

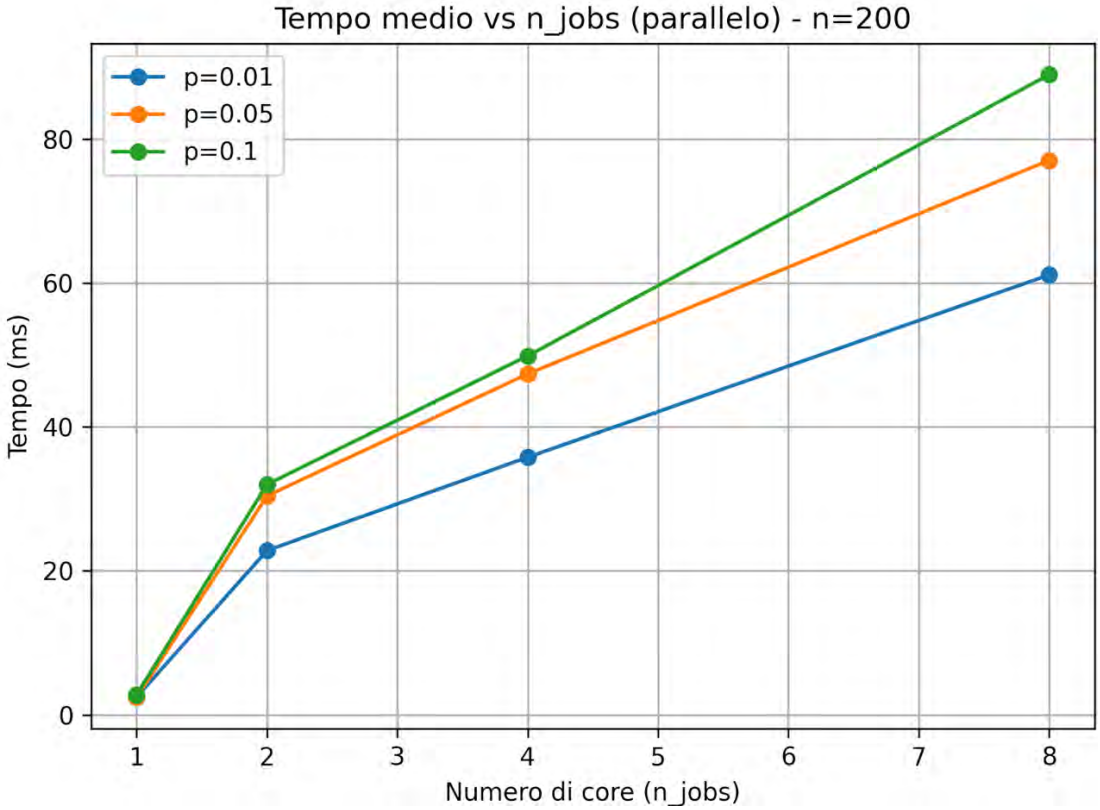
Questo andamento è particolarmente evidente considerando la forma delle curve: invece di crescere linearmente, esse mostrano incrementi via via più ridotti al crescere di n. Tale comportamento conferma sperimentalmente la complessità teorica O(log n) di Luby. È proprio questa crescita estremamente lenta del numero di round (e quindi del numero di barriere di sincronizzazione) a rendere l'algoritmo un candidato ideale per la scalabilità su sistemi HPC.

Un secondo aspetto osservabile riguarda l'effetto della densità del grafo. Le curve corrispondenti a valori più elevati di p si collocano sistematicamente al di sopra delle altre, indicando un numero di round leggermente maggiore. Questo suggerisce che, nei grafi più densi, il processo di eliminazione dei nodi per round è meno efficace, rendendo necessarie più iterazioni per completare l'algoritmo.

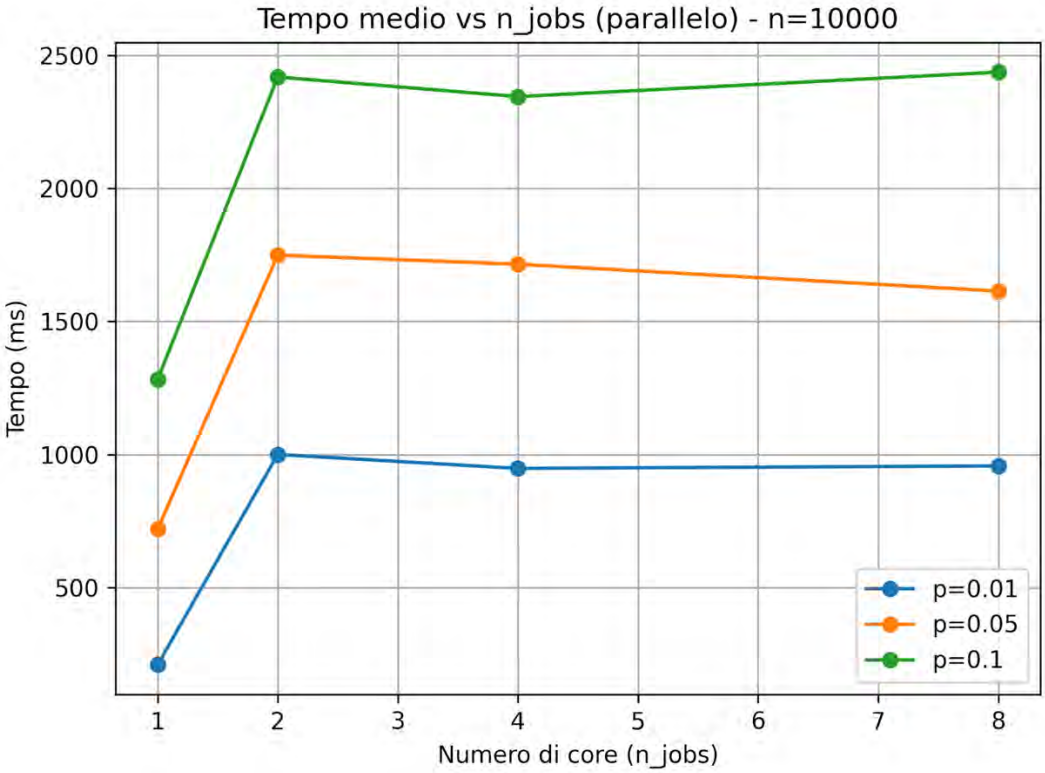
Si può inoltre notare che, per valori molto grandi di n, sono presenti solo alcune curve (quelle con p più piccolo). Questo riflette una scelta sperimentale: grafi molto grandi e densi comporterebbero costi computazionali e di memoria troppo elevati per essere gestiti nel contesto del cluster utilizzato.

4.4 Scalabilità rispetto al numero di core

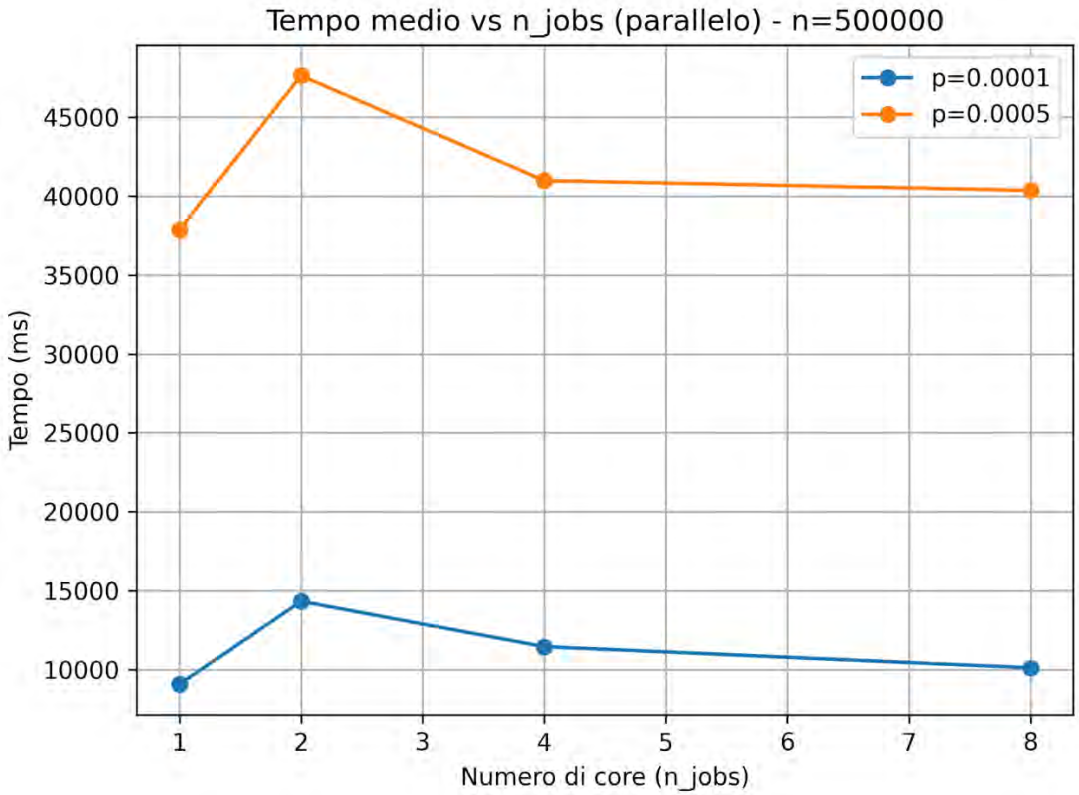
Per analizzare la scalabilità è stato valutato il tempo di esecuzione della versione parallela al variare del numero di core.



Tempo medio di esecuzione dell'algoritmo parallelo in funzione del numero di core per n=200.



Tempo medio di esecuzione dell'algoritmo parallelo in funzione del numero di core per n=10000n = 10000.



Tempo medio di esecuzione dell'algoritmo parallelo in funzione del numero di core per n=500000n = 500000.

Dall'analisi dei grafici emergono comportamenti differenti a seconda della dimensione del grafo, che mettono in evidenza il ruolo dominante dell'overhead di parallelizzazione.

Nel caso dei grafi piccoli (n=200), l'andamento è particolarmente chiaro: il tempo di esecuzione cresce quasi linearmente con il numero di core. Invece di osservare un miglioramento, l'aggiunta di worker introduce un costo aggiuntivo dovuto alla gestione del parallelismo (creazione dei task, sincronizzazione e comunicazione), che risulta superiore al lavoro computazionale effettivo. In questo regime, il parallelismo è quindi controproducente.

Per grafi di dimensione intermedia (n=10000), il comportamento cambia leggermente ma senza portare benefici significativi. Il passaggio da 1 a 2 core comporta un peggioramento netto delle prestazioni, mentre per valori più elevati di n_jobs il tempo tende a stabilizzarsi. Questo suggerisce che il costo iniziale del parallelismo rimane elevato, mentre l'aumento del numero di worker non riesce a compensarlo con un'effettiva riduzione del lavoro per processo. Questo risultato evidenzia come l'attivazione della memoria condivisa e la serializzazione degli oggetti abbiano un costo fisso che il carico di lavoro di 10.000 nodi non riesce ancora ad ammortizzare.

Nel caso dei grafi più grandi (n=500000), emerge un comportamento più interessante. Dopo un aumento iniziale del tempo passando da 1 a 2 core, si osserva una leggera diminuzione per valori maggiori di n_jobs. Questo indica che, per problemi sufficientemente grandi, il lavoro computazionale per round diventa abbastanza consistente da iniziare a compensare l'overhead.

Tuttavia, il miglioramento rimane limitato e lontano da uno speedup ideale. In particolare, nel caso in esame, il lavoro per singolo task è relativamente ridotto e frammentato su più round, rendendo il parallelismo meno efficace rispetto ai casi "embarrassingly parallel" analizzati da Boschi e Ali. ((Boschi & Ali, 2025))

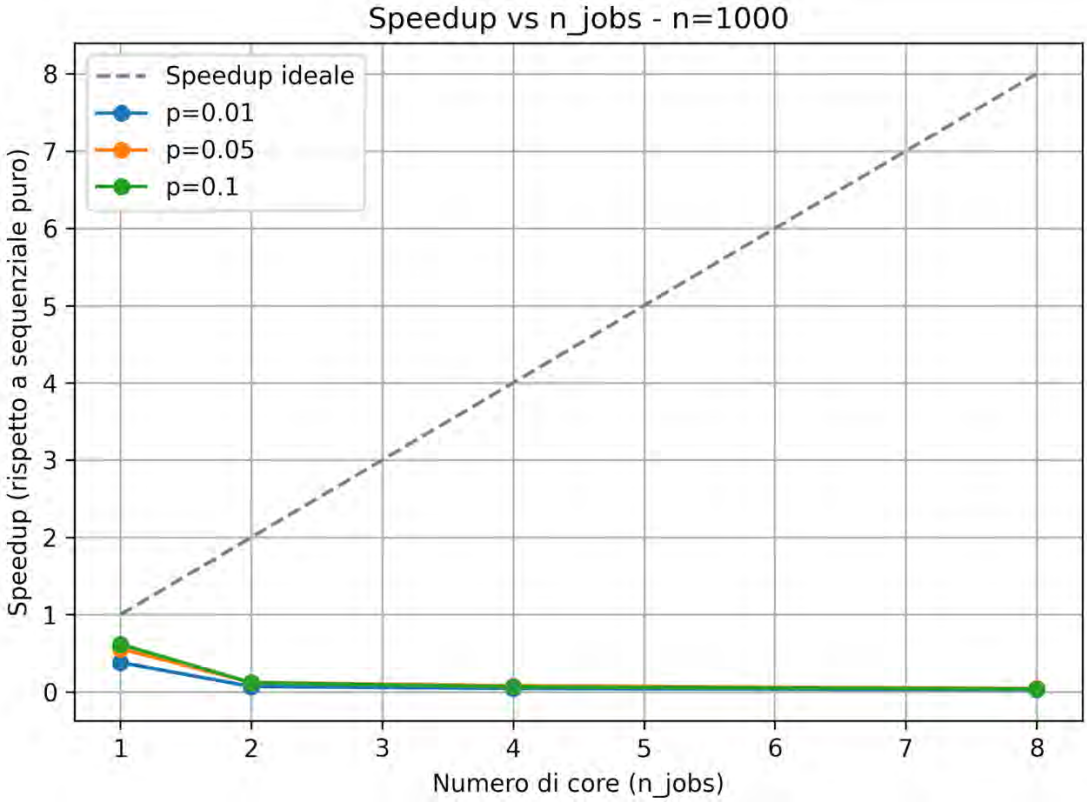
4.5 Efficienza e Legge di Amdahl

Per valutare l'efficacia della parallelizzazione è stato analizzato lo speedup:

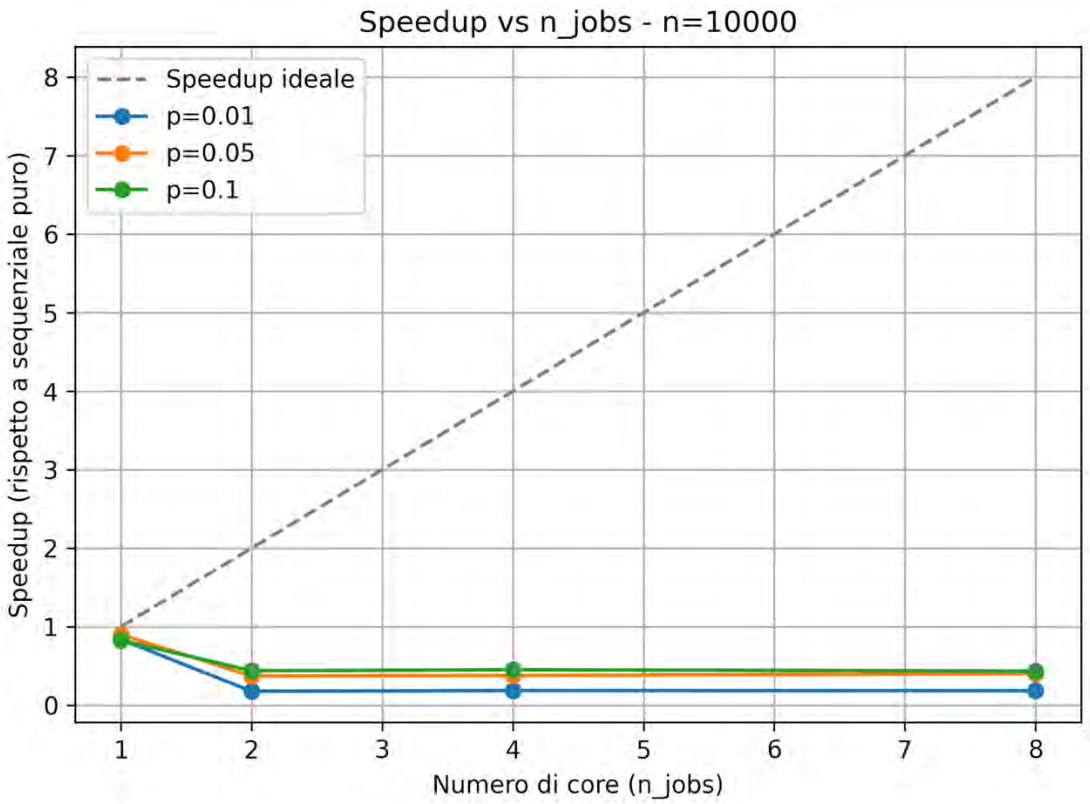
$$S = \frac{Tseq}{Tpar}$$

e l'efficienza:

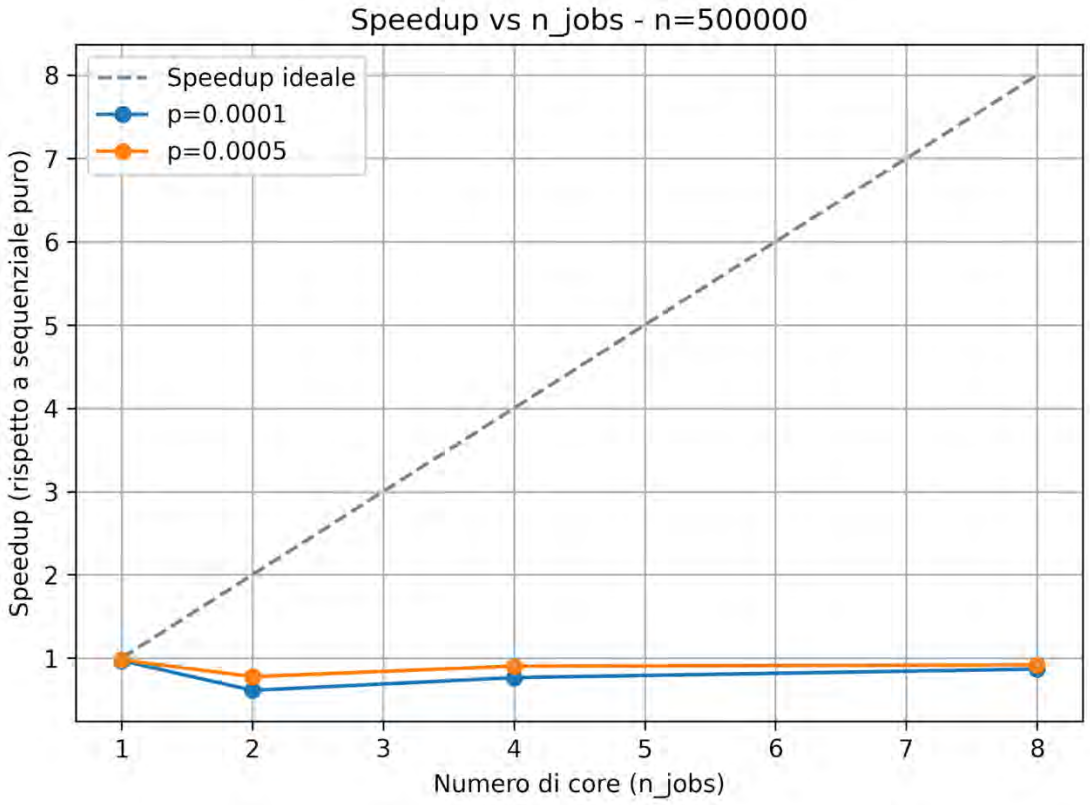
$$E = \frac{S}{n_jobs}$$



Speedup della versione parallela al variare del numero di core per n=1000



Speedup della versione parallela al variare del numero di core per n=10000



Speedup della versione parallela al variare del numero di core per n=500000

Dall'analisi dei grafici di speedup emerge chiaramente che la parallelizzazione dell'algoritmo non produce benefici significativi nella maggior parte delle configurazioni considerate.

Nel caso di grafi piccoli (n=1000), lo speedup è estremamente ridotto e diminuisce rapidamente all'aumentare del numero di core. Già con n_jobs si osserva un forte peggioramento, e per valori maggiori lo speedup si avvicina a zero. Questo comportamento indica che il tempo di esecuzione parallelo è dominato quasi interamente dall'overhead, che supera di gran lunga il lavoro utile. In questo scenario, l'Efficienza Parallela E tende allo 0%, indicando un totale spreco di risorse computazionali e confermando l'inefficacia del parallelismo per task a grana troppo fine.

Per grafi di dimensione intermedia (n=10000), la situazione migliora solo marginalmente. Lo speedup si stabilizza su valori compresi tra circa 0.2 e 0.4, senza mostrare una crescita significativa al crescere del numero di core. Questo suggerisce che, pur aumentando la quantità di lavoro per round, la frazione effettivamente parallelizzabile resta limitata e non consente di ottenere benefici concreti dalla parallelizzazione.

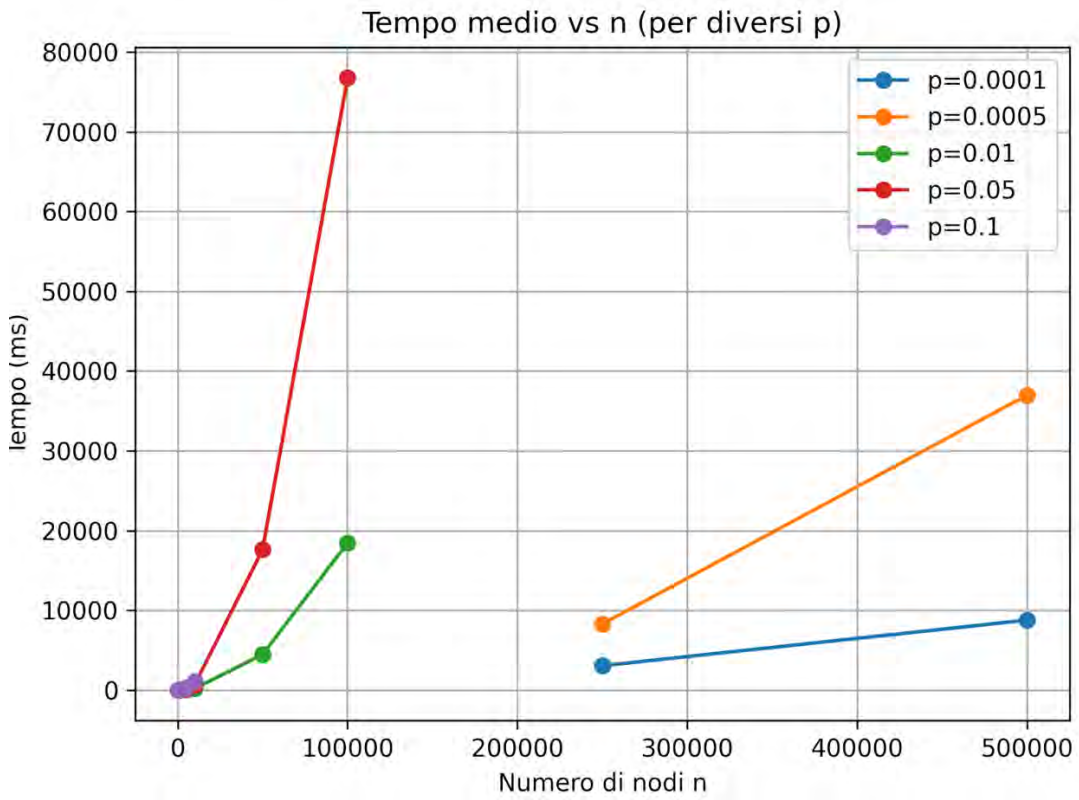
Un comportamento diverso si osserva per grafi molto grandi (n=500000). In questo caso lo speedup cresce sensibilmente, raggiungendo valori prossimi a 0.9 per un numero elevato di core. Ciò indica che, per problemi di dimensione sufficientemente grande, il costo computazionale per round diventa dominante rispetto all'overhead, permettendo una migliore utilizzazione delle risorse parallele. Tuttavia, anche in questo scenario non si raggiunge mai uno speedup superiore a 1, segno che la versione parallela non supera comunque quella sequenziale. I risultati ottenuti sono coerenti con quanto osservato da Boschi e Ali, che evidenziano come l'efficacia del parallelismo con Joblib dipenda fortemente dalla granularità dei task e dall'overhead di gestione, che può dominare il tempo di esecuzione per carichi computazionali ridotti. ((Boschi & Ali, 2025))

Questi risultati possono essere interpretati alla luce della legge di Amdahl, la quale definisce il limite teorico dello speedup in presenza di una frazione intrinsecamente sequenziale (1-f) nel codice: $S_max \leq \frac{1}{(1-f)+f/n_jobs}$. Nel caso in esame, i grafici confermano che tale frazione non è trascurabile: oltre all'overhead introdotto da Joblib, la fase di *Update* dell'algoritmo rimane rigorosamente sequenziale. Per evitare *race conditions* e garantire la correttezza del MIS, l'aggiornamento della maschera dei nodi attivi (active_mask) deve avvenire obbligatoriamente su un singolo processo al termine di ogni iterazione.

Di conseguenza, anche aumentando il numero di core, il guadagno ottenibile resta limitato. Solo per valori molto grandi di n l'impatto relativo dell'overhead diminuisce, consentendo di avvicinarsi leggermente al comportamento ideale, ma senza mai raggiungerlo. Questo evidenzia come, nel caso dell'algoritmo di Luby, la struttura iterativa e le dipendenze tra round riducano ulteriormente la frazione parallelizzabile rispetto a problemi completamente indipendenti. L'impatto ineludibile di questo collo di bottiglia seriale pone di fatto un asintoto orizzontale allo speedup massimo ottenibile, giustificando il crollo sistematico dell'efficienza all'aumentare dei processori.

4.6 Impatto della densità del grafo

Un altro fattore che influisce sulle prestazioni dell'algoritmo è la densità del grafo, determinata dalla probabilità **p** di presenza degli archi.



Tempo medio dell'implementazione sequenziale in funzione del numero di nodi del grafo

Osservando il grafico, si nota che il tempo di esecuzione cresce rapidamente all'aumentare del numero di nodi n, ma anche che tale crescita dipende in modo significativo dalla densità del grafo.

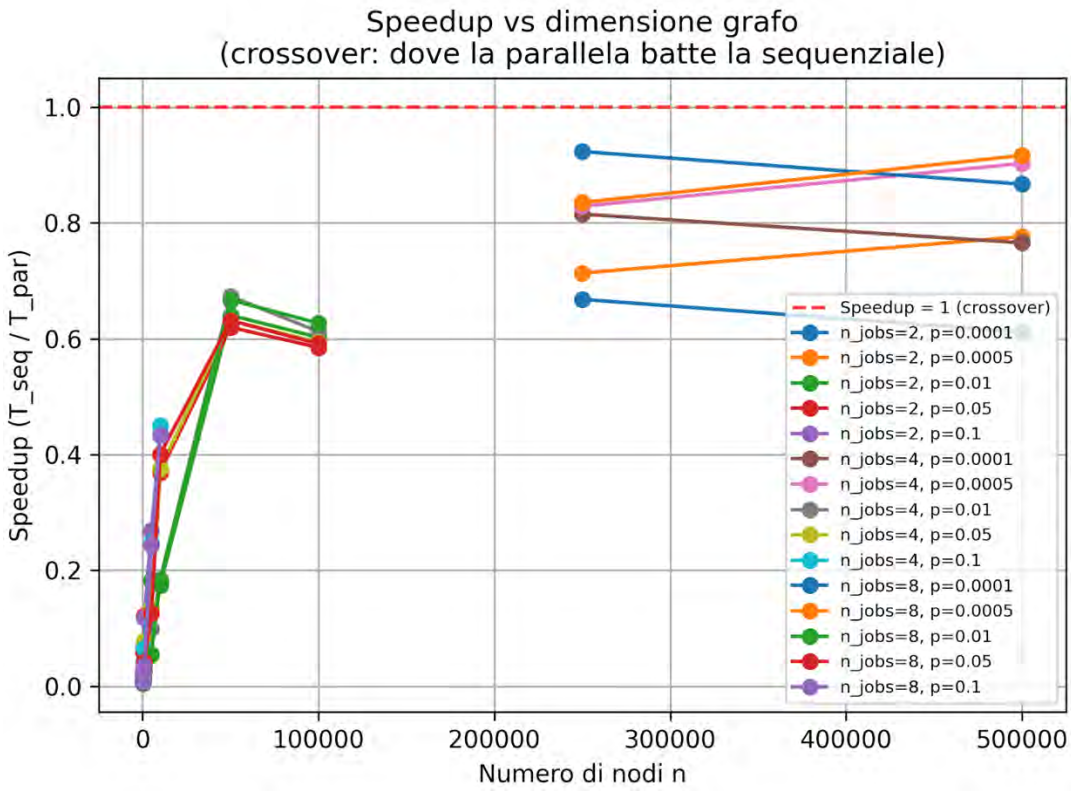
Le curve corrispondenti a valori diversi di p mostrano infatti comportamenti nettamente distinti: per probabilità molto basse (ad esempio p=0.0001), il tempo cresce in modo relativamente contenuto anche per grafi di grandi dimensioni, mentre per valori più elevati di p si osserva un incremento molto più marcato. In particolare, già per dimensioni intermedie del grafo, le curve associate a p=0.05 e p=0.1 crescono rapidamente, indicando un aumento significativo del costo computazionale.

Questo comportamento è direttamente collegato alla struttura del grafo. All'aumentare di p, il numero medio di archi cresce quadraticamente con n, e di conseguenza aumenta il grado medio dei nodi. Poiché in ogni round ciascun nodo deve confrontare la propria priorità con quella di tutti i suoi vicini, un maggiore grado implica un numero più elevato di confronti, e quindi un costo computazionale più alto per ogni iterazione dell'algoritmo.

Un aspetto interessante che emerge dal grafico è che, per grafi molto grandi, sono stati considerati solo valori piccoli di p. Questo riflette un limite pratico della sperimentazione: grafi molto densi e di grandi dimensioni comportano un numero di archi estremamente elevato, rendendo proibitivi sia il tempo di esecuzione sia l'uso della memoria.

4.7 Quando NON conviene parallelizzare Luby

Per concludere l'analisi, è stato effettuato un confronto diretto tra i tempi della versione sequenziale e quelli della versione parallela utilizzando il numero massimo di worker disponibili.



Confronto delle prestazioni tra implementazione sequenziale e parallela dell'algoritmo di Luby per grafi di dimensione crescente.

Osservando il grafico, emerge chiaramente come la relazione tra versione sequenziale e parallela dipenda fortemente dalla dimensione del grafo.

Per grafi di piccole dimensioni, lo speedup assume valori molto bassi, spesso prossimi a zero. In questa regione, la versione parallela risulta sistematicamente più lenta di quella sequenziale, indicando che il costo introdotto dalla gestione del parallelismo supera ampiamente il beneficio computazionale.

All'aumentare della dimensione del grafo, si osserva una progressiva crescita dello speedup. Le curve mostrano un miglioramento continuo, che diventa particolarmente evidente per valori elevati di n, dove lo speedup raggiunge valori compresi tra circa 0.6 e 0.9. Questo andamento indica che il parallelismo inizia a diventare più efficace quando il lavoro computazionale complessivo è sufficientemente grande.

Tuttavia, un aspetto fondamentale emerge dall'osservazione della linea di riferimento corrispondente a S=1S = 1S=1, che rappresenta il punto di crossover tra versione sequenziale e parallela. Nessuna delle curve raggiunge o supera questa soglia nell'intervallo sperimentale considerato. In altre parole, la versione parallela non raggiunge il **punto di crossover** entro il range sperimentale testato.

Questo risultato suggerisce che, nel contesto analizzato, il punto di crossover si colloca oltre i valori di n considerati, ovvero per grafi significativamente più grandi di quelli utilizzati negli esperimenti. Di conseguenza, per tutte le configurazioni testate, l'utilizzo del parallelismo non risulta conveniente dal punto di vista delle prestazioni.

Nel complesso, il grafico evidenzia come l'efficacia della parallelizzazione non sia una proprietà intrinseca dell'algoritmo, ma dipenda fortemente dalla dimensione del problema e dai costi pratici dell'implementazione. Nel caso specifico, tali costi rimangono dominanti anche per grafi di grandi dimensioni, impedendo di ottenere un reale vantaggio rispetto alla versione sequenziale.

5. Conclusioni e sviluppi futuri

L'analisi sperimentale ha evidenziato che, nelle configurazioni considerate, la versione parallela dell'algoritmo di Luby non riesce a superare la versione sequenziale in termini di tempo di esecuzione. In particolare, per grafi di piccole e medie dimensioni, l'utilizzo di più core comporta spesso un peggioramento delle prestazioni, dovuto al costo aggiuntivo introdotto dalla gestione del parallelismo.

All'aumentare della dimensione del grafo, si osserva tuttavia un cambiamento nel comportamento: lo speedup cresce progressivamente e, per grafi molto grandi, si avvicina a valori prossimi a 1. Questo indica che l'overhead di parallelizzazione diventa relativamente meno significativo rispetto al lavoro computazionale complessivo. Nonostante ciò, il punto di crossover (ovvero la condizione in cui la versione parallela diventa più veloce di quella sequenziale) non viene mai raggiunto nell'intervallo sperimentale analizzato.

Questi risultati sono coerenti con le caratteristiche strutturali dell'algoritmo di Luby. Sebbene il problema del Maximal Independent Set sia teoricamente altamente parallelizzabile, l'algoritmo presenta una dipendenza iterativa tra i round: ogni iterazione richiede l'aggiornamento completo del grafo residuo prima di poter procedere alla successiva. Di conseguenza, il parallelismo è limitato alla sola fase di selezione dei minimi locali all'interno di ciascun round, riducendo la frazione effettivamente parallelizzabile.

L'analisi è inoltre coerente con la legge di Amdahl, secondo cui la presenza di una componente sequenziale e di costi di coordinamento limita lo speedup massimo ottenibile. Nel caso in esame, tali limiti (teorizzati dalla Legge di Amdahl) sono stati mitigati, ma non eliminati, dalle scelte architetturali adottate: sebbene la conversione del grafo in formato **CSR-like (NumPy)** e l'uso della **memoria condivisa (memmap)** abbiano evitato i crash per *Out-of-Memory*, l'uso di Python (e del backend *loky*) per un problema a granularità fine introduce latenze insopprimibili.

Il confronto con lavori presenti in letteratura, come quello di Boschi e Ali, evidenzia ulteriormente questo aspetto. Nei loro esperimenti, il parallelismo risulta efficace per task indipendenti (embarrassingly parallel), mentre nel caso dell'algoritmo di Luby la presenza di dipendenze tra le iterazioni riduce significativamente l'efficacia della parallelizzazione.

Nel complesso, il progetto ha permesso di mettere in evidenza un aspetto fondamentale del calcolo ad alte prestazioni: la parallelizzazione non garantisce automaticamente un miglioramento delle prestazioni, ma richiede un'attenta valutazione del rapporto tra costo di coordinamento e lavoro computazionale. In particolare, affinché il parallelismo sia efficace, è necessario che la granularità del problema sia sufficientemente elevata da compensare l'overhead introdotto.

Per quanto riguarda gli sviluppi futuri, un primo possibile miglioramento consiste nell'estendere la sperimentazione a grafi di dimensioni ancora maggiori, al fine di verificare se il punto di crossover possa essere effettivamente raggiunto per valori di n più elevati. Inoltre, sarebbe cruciale valutare l'utilizzo di paradigmi di parallelizzazione nativi. Da un lato, il passaggio a **C++ con OpenMP o MPI** permetterebbe di abbattere l'overhead del thread-management. Dall'altro, considerando che la

ricerca dei minimi locali è un'operazione vettoriale massivamente parallela, l'algoritmo di Luby si presterebbe idealmente al porting su architetture a molti core come le **GPU (es. tramite CUDA o CuPy)**, dove l'elevata banda passante della memoria risolverebbe il limite strutturale del *Memory-Bound*.

Un'ulteriore direzione riguarda l'adozione di rappresentazioni dei dati e strutture più ottimizzate per l'accesso in memoria, che potrebbero migliorare le prestazioni complessive dell'algoritmo anche in contesto parallelo.

In conclusione, l'algoritmo di Luby rappresenta un esempio paradigmatico di algoritmo teoricamente efficiente in ambito parallelo, ma la sua implementazione pratica evidenzia chiaramente come le prestazioni reali dipendano in modo critico dal modello computazionale, dagli strumenti utilizzati e dalla dimensione del problema.

Appendice A – Esecuzione degli esperimenti

A.1 Test in ambiente locale

Le verifiche preliminari di correttezza sono state eseguite in ambiente locale, su grafi di piccole dimensioni, al fine di validare l'implementazione prima dell'esecuzione dei benchmark su cluster.

In particolare, sono stati eseguiti i seguenti passi:

```
# Verifica struttura pacchetto
find . -maxdepth 2 -type f -name "__init__.py" | sort

# Controllo sintattico
python -m py_compile $(find src -type f -name "*.py")

# Test di correttezza
PYTHONPATH=. python -m src.sequential.tests_seq
PYTHONPATH=. python -m src.parallel.tests_par
```

A.2 Esecuzione su cluster HPC

I benchmark prestazionali sono stati eseguiti sul cluster HPC dell'Università di Parma con l'accesso alla VPN.

Accesso e trasferimento file

```
ssh nome.cognome@login.hpc.unipr.it
scp -r src nome.cognome@login.hpc.unipr.it:~
```

Installazione dipendenze

```
python3 -m pip install --user numpy joblib pandas matplotlib
```

Test di correttezza su cluster

```
python3 -m src.sequential.tests_seq
python3 -m src.parallel.tests_par
```

A.3 Script SLURM per benchmark

I benchmark sono stati eseguiti tramite il seguente script SLURM:

```
#!/bin/bash
#SBATCH --job-name=Luby_nome
#SBATCH --output=risultati_nome.log
#SBATCH --error=errori_nome.log
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --time=04:00:00

cd $HOME
export PYTHONPATH=.

echo "--- AVVIO TEST DI CORRETTEZZA ---"
python3 -m src.sequential.tests_seq
python3 -m src.parallel.tests_par

echo "--- AVVIO BENCHMARK SEQUENZIALE ---"
python3 -m src.sequential.bench_seq

echo "--- AVVIO BENCHMARK PARALLELO ---"
python3 -m src.parallel.bench_par

echo "--- GENERAZIONE GRAFICI ---"
python3 -m src.sequential.plot_seq
python3 -m src.parallel.plot_par
python3 -m src.parallel.plot_compare
```

Esecuzione del job:

```
sbatch run_luby.sh
squeue -u nome.cognome
```

A.4 Raccolta risultati

I risultati sono stati trasferiti dal cluster al sistema locale:

```
Scp nome.cognome@login.hpc.unipr.it:~/par_results.csv .
scp nome.cognome@login.hpc.unipr.it:~/seq_results.csv .
scp -r nome.cognome@login.hpc.unipr.it:~/plots_* .
```

Bibliografia

[Online] / auth. jobs Joblib: running Python functions as pipeline. - https://joblib.readthedocs.io/en/stable/.

A Simple Parallel Algorithm for the Maximal Independent Set Problem [Online] / auth. Luby Michael // SIAM Journal on Computing. - https://share.google/7YuQoXdJnzDfABORo.
Analisi Critica del Parallelismo con Joblib su Cluster HPC [Online] / auth. Giada Boschi Nida Ali.
Luby's Algorithm for Maximal Independent Set [Online] / auth. Miller Gary // Carnegie Mellon University. - https://share.google/npNsFdZkVGiQkXcIB.
Maximal Independent Set – Distributed Computing [Online] / auth. Group Distributed Computing // ETH Zürich. - https://share.google/HtDNlCOmEi0wYgT6I.